

RETRO SNAKE GAME — v2.0

Maintenance Report

CSE444 Software Engineering II

Homework: Vibe Coding Game Development Project Maintenance

1. Introduction

Original project: The v1.0 Retro Snake Game is a Python/PyGame implementation of the classic Snake arcade game on a 40×29 grid, featuring normal food (+10 pts), timed bonus gold food (+30 pts), delta-time movement, particle effects, a colour-gradient snake body, and JSON high-score persistence.

Maintenance objectives: Extend the game with (1) 12-level named difficulty scaling for both player and robot speeds, (2) an optional robot snake hazard, (3) three special food collectables with distinct effects, (4) an independent timer-based food spawn system, (5) improved food placement to avoid all occupied cells, and (6) gameplay refinements to bonus, poison, and slow-motion food — all while preserving original functionality.

2. Functional Requirements

Original system: 25 requirements covering grid display, controls, movement, food, collision, scoring, speed, HUD, pause, game-over, and persistence were preserved or evolved without breaking the core gameplay. New requirements added in v2.0:

ID	Requirement
FR-26	Display 12 named difficulty levels (ROOKIE → GODLIKE); player and robot speed scale at score thresholds.
FR-27	Show level name, colour, and a progress bar toward the next level in the HUD.
FR-28	Display a 'LEVEL UP!' banner with fade-out when crossing a level threshold.
FR-29	B key toggles the robot snake on/off (in menu and during play).
FR-30	Robot snake wanders randomly, avoiding walls; does not chase food or player.
FR-31	Player head hitting robot body ends the game (unless invincible).
FR-32	Robot head hitting player body makes the robot disappear for 30 s, then respawn safely.
FR-33	Robot speed scales with the current difficulty level in real time.
FR-34	Robot snake consumes food it moves onto; no points or effects gained.
FR-35	Special foods spawn on an independent 5-second timer (50% chance per check).
FR-36	Up to 3 special foods may be active simultaneously.
FR-37	All food placements avoid player body, robot body, and all other food positions.
FR-38	[I] Cyan Invincibility food: wall wrap + self/robot immunity for 10 s; refreshes on re-eat.
FR-39	[P] Purple Poison food: removes up to 3 tail segments, deducts 10 pts/segment (min score 0).
FR-40	[S] Orange Slow-motion food: reduces speed to ~45% for 10 s; refreshes to full duration on re-eat.
FR-41	Bonus gold food now grants +3 body segments in addition to +30 points.
FR-42	When robot is added mid-game, existing foods relocate if they overlap the robot body.

3. Approach to the Problem — Vibe Coding

3.1 Analysing the Codebase

Before writing any new code, the existing architecture was mapped by reading the five classes top-to-bottom. Three integration points were identified: *Game._do_move()* (all collision and scoring), *Game._update()* (all per-frame timers), and the food constructors (cell placement). This analysis guided every subsequent change to avoid unnecessary rewrites.

3.2 Iterative Development Steps

- **Independent food spawning:** Added *_special_spawn_timer* in *_update()* calling *_try_spawn_special_food()* every 5 s. Decoupled special food appearance from player actions; adjusted *SPECIAL_CHANCE=0.5* and *MAX_SPECIAL_FOODS=3* through play-testing.
- **Multi-food list:** Replaced single *self.special_food* with *self.special_foods* list. Updated draw/update/collision loops to iterate the list. Zero architecture changes needed.
- **Food placement system:** Introduced *_food_blockers()* helper returning all occupied non-player cells (robot body + existing foods). All constructors gained an optional *extra_occupied* parameter with *None* default — existing call sites unchanged.
- **Robot snake:** RobotSnake class added with random wall-safe movement, 30-second hide mechanic, and per-level speed sync in *_update()*. *_robot_eat_foods()* handles food consumption without giving the robot score or effects.
- **Gameplay refinements:** Bonus food: added 3 *grow()* calls. Poison: added -10 pts/segment with score floor at 0 and level re-sync. Slow-motion: uses a fixed *SLOWMO_DURATION* timer and refreshes to full duration when re-eaten. Invincibility: wall wrap via *nc%COLS*.
- **12-level system:** Expanded *DIFFICULTY_LEVELS* from 7 to 12 rows. All consumers (*_get_difficulty*, *_level_progress*) required zero changes.

3.3 AI Agent Usage — Vibe Coding with Claude

Throughout this maintenance project, the Claude (Anthropic) AI assistant was used as the primary development agent — consistent with the vibe coding methodology described in the assignment brief. Rather than planning every feature upfront, the process involved iterative dialogue with the AI agent: describing a requirement, reviewing the proposed change, testing mentally or in-game, and feeding the result back into the next prompt.

The AI agent contributed at four distinct levels:

- **Code analysis:** The agent read and summarised the existing codebase (Snake, Food, BonusFood, Particle, Game), identified the three natural integration points (*_do_move*, *_update*, food constructors), and flagged potential conflict zones before any change was made.
- **Incremental code generation:** Each new feature (RobotSnake, SpecialFood subclasses, *_food_blockers*, *_try_spawn_special_food*, level-up banner) was generated in a targeted diff-style approach. The agent was explicitly instructed not to rewrite the whole file, preserving original logic paths.
- **Bug detection and fixing:** The agent independently identified edge cases — *ZeroDivisionError* at max level (*span=0*), foods spawning on each other (fixed by *_food_blockers*), poison making the score negative (fixed with *max(0, score - loss)*), and robot spawning on existing food (fixed by *_relocate_foods_if_needed*) — and proposed targeted fixes.
- **Documentation generation:** The README, functional requirements table, and this report were drafted by the AI agent based on the implemented code, then reviewed and approved by the developer.

The three examples below show the developer–AI dialogue in practice:

Example 1 — Fixing the Max-Level Crash

Developer prompt:

“When the score reaches the final level, the game crashes with ZeroDivisionError in _level_progress(). It seems to happen around the maximum level.”

AI agent response:

“The crash is caused by the progress-bar calculation trying to divide by zero when there is no next level. The fix is to detect when the current level is the last entry in DIFFICULTY_LEVELS and return (1.0, None) instead of calculating a span.”

Implemented change:

The `_level_progress()` method was updated to use an explicit `current_index` check. When the player reaches the final level, the HUD now displays MAX instead of trying to calculate progress toward a non-existing next level. This solved the runtime crash without affecting the difficulty system.

Example 2 — Supporting Multiple Special Foods

Developer prompt:

“Only one special food can exist at a time. I want multiple special foods to appear on the board simultaneously.”

AI agent response:

“The current design uses a single `self.special_food` reference, which limits the system to one active collectable. Replace it with a `self.special_foods` list and update the draw, update, player-collision, and robot-food interaction logic to iterate over that list. Add `MAX_SPECIAL_FOODS` to keep the board balanced.”

Implemented change:

The special food system was refactored from a single-object model to a list-based model. This allowed up to three active special foods at the same time while preserving clear update and collision logic.

Example 3 — Preventing Food Overlap

Developer prompt:

“Special foods and normal foods can appear on top of each other. No food should spawn on the player snake, robot snake, or another food.”

AI agent response:

“Centralise all occupied-cell logic in a `_food_blockers()` helper. It should include the player snake, robot snake, normal food, bonus food, and active special foods. Then pass this list to `Food`, `BonusFood`, and `SpecialFood` through an `extra_occupied` parameter.”

Implemented change:

Food spawning was made safer and more consistent. All food types now avoid player body cells, robot body cells, and existing food positions.

All architectural decisions — which features to add, how much to refactor, which bugs to prioritise — were made by the developer. The AI agent acted as a knowledgeable pair-programmer: it could propose and generate, but it did not run, test, or deploy code autonomously. Final validation (syntax checks, play-testing, compatibility review) was always performed by the developer.

This workflow directly reflects the vibe coding principle: momentum and rapid iteration over exhaustive upfront planning. The AI agent made it feasible to explore and implement six distinct maintenance tasks within the one-week timeframe while maintaining code quality and backward compatibility with all 25 original functional requirements.

3.4 Challenges and Solutions

- **ZeroDivisionError at max level:** span=0 when at last DIFFICULTY_LEVELS entry. Fixed by computing next_min with an explicit index check, returning (1.0, None) at last level.
- **Foods spawning on each other:** free_cells(snake.body) insufficient with robot + multi-food. _food_blockers() now aggregates all occupied positions passed to every constructor.
- **Robot spawning on food mid-game:** _relocate_foods_if_needed() called immediately after B-key robot creation to move conflicting foods.
- **Poison making score negative:** Added score = max(0, score – loss) and re-synced _prev_level to prevent false level-up triggers after score drop.

4. Implementation Overview

4.1 Component Summary

Component	Change	Details
Snake	Extended	move(invincible=False): wall wrap + self-collision skip
Food / BonusFood	Extended	extra_occupied param; BonusFood now +3 segments
SpecialFood base	Extended	extra_occupied param; TTL 12 s
InvincibilityFood	Extended	Refreshes timer on re-eat
PoisonFood	Extended	-10 pts/segment; score ≥ 0; level re-sync
SlowMotionFood	Extended	Refreshes to full duration on re-eat
RobotSnake	New	Random wander; hide 30 s; speed from difficulty table
Game	Extended	special_foods list; _food_blockers; _try_spawn_special_food; _robot_eat_foods; 12-level table; level-up banner

4.2 Difficulty Level Table

L#	Name	Min Score	Player spd	Robot spd
L1	ROOKIE	0	7.0	3.5
L2	TRAINEE	40	7.5	4.0
L3	NOVICE	90	8.0	4.5
L4	APPRENTICE	150	8.6	5.0
L5	SKILLED	230	9.2	5.8
L6	ADVANCED	330	9.9	6.5
L7	EXPERT	450	10.7	7.3
L8	MASTER	600	11.5	8.0
L9	ELITE	780	12.4	8.8
L10	LEGEND	1000	13.3	9.6
L11	MYTHIC	1250	14.2	10.5
L12	GODLIKE	1550	15.0	11.5

4.3 Tools and Technologies

Python 3.13 | PyGame 2.6.1 (graphics, input, timing) | reportlab 4.x (this report) | json / math / random (stdlib) | AI assistant — code generation, review, debugging, and documentation support

5. Discussions and Reflections

5.1 Key Technical Changes

- **special_food** → **special_foods list**: multiple simultaneous collectables with zero logic restructuring.
- **_food_blockers()** centralises spawn-exclusion across all 5 food spawn sites.
- **_try_spawn_special_food()** decouples food appearance from player actions (timer-driven).
- **_robot_eat_foods()** adds board interaction without touching player collision logic.
- **Slow-motion timer refresh**: re-eating slow-motion food resets the effect to full duration, keeping the mechanic simple and balanced.
- **12-level DIFFICULTY_LEVELS table**: `_get_difficulty()` and `_level_progress()` required zero changes.
- **AI agent integration**: all six maintenance tasks delivered within one week using iterative AI-assisted development, with backward compatibility verified at each step.

5.2 Compatibility with Original System

All 25 original functional requirements remain supported, while some behaviours were evolved for the maintenance version without breaking the core gameplay. Constructors gained optional parameters (default None) so existing call sites never changed. The state machine, controls, HUD structure, JSON high-score format, and visual style are preserved. New colours (cyan, purple, orange, red robot) are visually distinct from the original palette.

5.3 Lessons Learned

- **Reading code before writing**: mapping `_do_move()` first saved hours of debugging by making integration points obvious.
- **Optional parameters with None defaults** extend constructors without breaking existing call sites.
- **Single-responsibility helpers** keep `_update()` readable despite significant new behaviour.
- **Constants tuned through play** (`SPECIAL_SPAWN_INTERVAL`, `MAX_SPECIAL_FOODS`) produce better results than upfront calculation.
- **Effect timer behaviour must match gameplay balance**: invincibility and slow-motion both refresh to full duration when collected again.
- **AI-assisted vibe coding works best** when the developer retains decision authority: the agent proposes, the developer validates and integrates. This division of labour accelerates implementation without sacrificing code ownership or understanding.

5.4 Maintainability, Extensibility & Future Improvements

Adding a fourth special food type requires only a new `SpecialFood` subclass and one `isinstance()` branch. Adding a new level requires one tuple in `DIFFICULTY_LEVELS`. The `_food_blockers()` pattern means any future entity registers its cells there without touching food constructors. Planned improvements: sound effects (8-bit beeps), a second robot at L10+, static obstacle tiles at L6+, and a top-5 session leaderboard.